

WasteVision: AI-Powered Waste Recognition Using YOLOv8 and OpenCV

Project Aim

WasteVision is a computer vision-based waste classification system that uses YOLOv8 and OpenCV to recognise and determine different types of waste with a given image or video feed. This project aims to:

- To train a model using a dataset to improve accuracy during detection
- To create and develop a system that would be able to accurately identify different wastes using computer vision
- To develop a simple GUI that would make the system more interactive and accessible
- To raise environmental awareness
- To test and validate the system's performance through testing

Dependencies

The following python libraries below are required for the program to run:

- Ultralytics (YOLOv8) – Object detection model
- OpenCV – Image processing and video capture
- Tkinter – GUI for displaying results

How to run:

The program starts by first importing libraries it will need to run the program, as well as the trained YOLOv8 model.

```
import cv2
import tkinter as tk
from tkinter import filedialog, ttk
from PIL import Image, ImageTk
from ultralytics import YOLO

# YOLOv8 model
model = YOLO("runs/detect/train13/weights/best.pt")
```

It will then initialise the main menu which defines the window style (size and background color), the header with the program title, video frame where the video feed or image feed will be shown, and as well as the control buttons.

```
# Main window
root = tk.Tk()
root.title("Trash Detection System")
root.geometry("900x800") # Adjusted size
root.configure(bg="#2C3E50") # Dark theme
```

```

# Header Frame
header_frame = tk.Frame(root, bg="#1ABC9C", height=50)
header_frame.pack(fill="x")

title_label = tk.Label(header_frame,
                        text="Trash Detection System",
                        font=("Arial", 20, "bold"),
                        fg="white",
                        bg="#1ABC9C",
                        pady=10
                        )
title_label.pack()

# Video Frame (For image/webcam feed)
video_frame = tk.Frame(root, bg="#34495E", bd=5)
video_frame.pack(pady=20)

label_img = tk.Label(video_frame,
                      image=black_screen,
                      width=display_width,
                      height=display_height
                      )
label_img.pack()

```

The display in where the actual video feed will be displayed is now initialise along with its width, height and along with a black screen that will be applied whenever a display is not selected or off.

```

# Default display area size
display_width = 720
display_height = 480

# Function to create a blank black image
def create_black_image(width, height):
    black_img = Image.new("RGB", (width, height), "black")
    return ImageTk.PhotoImage(black_img)

# Default black screen
black_screen = create_black_image(display_width, display_height)

# Header Frame
header_frame = tk.Frame(root, bg="#1ABC9C", height=50)
header_frame.pack(fill="x")

```

Before the captured video feed or uploaded image gets displayed in the GUI. It first needs to be resized to fit the given display in the GUI.

```

# Function to resize image to fit display area while maintaining aspect ratio
def resize_image(image, max_width, max_height):
    img_width, img_height = image.size

    # Calculate scaling factor
    scale_factor = min(max_width / img_width, max_height / img_height)

    # Resize image
    new_width = int(img_width * scale_factor)
    new_height = int(img_height * scale_factor)

    return image.resize((new_width, new_height), Image.Resampling.LANCZOS)

```

If the user ever chooses to upload an image for waste detection. Two functions are used which are `upload_img()` and `detect_image(path)`. `upload_img()` uses `filedialog` to fetch chosen image file, then gets processed to the model using `detect_image(path)`

```
# Image Upload
def upload_img():
    file_path = filedialog.askopenfilename(filetypes=(("Image Files", "*.jpg;*.png;*.jpeg;*")))
    if file_path:
        detect_image(file_path)

# Image Detection
def detect_image(path):
    results = model(path)
    result_img = results[0].plot()

    img = cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB)
    img = Image.fromarray(img)
    img = resize_image(img, display_width, display_height) # Resize to fit
    img = ImageTk.PhotoImage(img)

    label_img.config(image=img)
    label_img.image = img
```

Otherwise, if webcam feed is chosen. A repeated function(`update_webcam()`) is called that process each frame given by the current chosen webcam. This function stops whenever cap isn't receiving a video feed.

```
# Function to update webcam feed inside GUI
def update_webcam():
    global cap
    if cap is None or not cap.isOpened():
        return

    ret, frame = cap.read()
    if not ret:
        return

    # Inference
    result = model(frame)
    predictions = result[0].boxes

    for box in predictions:
        x1, y1, x2, y2 = box.xyxy[0]
        confidence = box.conf[0]
        class_id = int(box.cls[0])

        cv2.rectangle(frame, (int(x1), int(y1)), (int(x2), int(y2)), (0, 255, 0), 2)
        class_name = model.names[class_id]
        cv2.putText(frame, f"{class_name} ({confidence:.2f})", (int(x1), int(y1) - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    # Convert OpenCV frame to Tkinter-compatible format
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    frame = Image.fromarray(frame)
    frame = resize_image(frame, display_width, display_height) # Resize to fit
    frame = ImageTk.PhotoImage(frame)

    label_img.config(image=frame)
    label_img.image = frame

    # Repeat function after 10ms
    root.after(10, update_webcam)
```

Inorder to start and stop webcam feed. Two functions will manage the state of the webcam which is `start_webcam()` and `stop_webcam()`: The `start_webcam()` function will first release any previously opened camera to avoid any conflicts. Afterwards will retrieve the selected camera index chosen by user and will now open selected camera and starts detection by continuously updating camera feed (`update_webcam()`). `stop_webcam` function will also release the current active camera and will reset the video display to black screen.

```

# Function to start webcam inside GUI
def start_webcam():
    global cap

    # Release existing camera if any
    if cap:
        cap.release()
        cap = None

    # Get selected camera index
    cam_index = selected_camera.get()

    cap = cv2.VideoCapture(cam_index)

    update_webcam()

# Function to stop webcam and reset to black screen
def stop_webcam():
    global cap
    if cap:
        cap.release()
        cap = None
        label_img.config(image=black_screen)
        label_img.image = black_screen

```

Each button is defined and makes up the main GUI. This includes buttons for uploading, starting and stopping webcam as well as which webcam to be used. This section also includes the styling of said buttons.

```

# Button Frame
btn_frame = tk.Frame(root,
                      bg="#2C3E50"
                      )
btn_frame.pack(pady=10)

# Buttons
btn_upload = tk.Button(btn_frame,
                       text="Upload Image",
                       command=upload_img,
                       **button_style
                       )
btn_upload.grid(row=0,
               column=0,
               padx=10,
               pady=5
               )

btn_webcam = tk.Button(btn_frame,
                      text="Start Webcam",
                      command=start_webcam,
                      **button_style
                      )
btn_webcam.grid(row=0,
               column=1,
               padx=10,
               pady=5
               )

btn_stop_webcam = tk.Button(btn_frame,
                           text="Stop Webcam",
                           command=stop_webcam,
                           **button_style
                           )
btn_stop_webcam.grid(row=0,
                    column=2,
                    padx=10,
                    pady=5
                    )

```

```

# Button Frame
btn_frame = tk.Frame(root,
                      bg="#2C3E50"
                      )
btn_frame.pack(pady=10)

# Buttons
btn_upload = tk.Button(btn_frame,
                      text="Upload Image",
                      command=upload_img,
                      **button_style
                      )
btn_upload.grid(row=0,
               column=0,
               padx=10,
               pady=5
               )

btn_webcam = tk.Button(btn_frame,
                      text="Start Webcam",
                      command=start_webcam,
                      **button_style
                      )
btn_webcam.grid(row=0,
               column=1,
               padx=10,
               pady=5
               )

btn_stop_webcam = tk.Button(btn_frame,
                           text="Stop Webcam",
                           command=stop_webcam,
                           **button_style
                           )
btn_stop_webcam.grid(row=0,
                    column=2,
                    padx=10,
                    pady=5
                    )

```